

THE STORY OF BEATWAVE

How We Built An Open-Source Music

App

By- Aarav Sharma , Akshansh
Sinha and Aarav Singh

Special thanks to-

We would like to thank our
parents our family and our
friends for tolerating our late

night clicks in the making of this app. To anyone who believes that great softwares should be built for the people , completely free, and entirely without annoying interruptive ads.

Acknowledgements

First and foremost, this application would not ever exist without Akshansh Sinha, As the lead developer of BeatWave, his brilliant problem-solving ,tireless attempts to build the apk , and all the dedication he put, pushing through all the broken builds turned a delusional conversation to reality . Thank you for being an exceptional partner in coding, a brilliant developer , and a true friend.

We extend our deepest gratitude to the global open-source and FOSS Community across GitHub, developer subreddits , and forum boards. We will also like to thank the developers of Echo Music and Convx. The rapid feedbacks, bug reports, and sheer enthusiasm from our users gave us the fuel to keep refining our code when errors almost made us close our laptops and quit.

Finally we thank our families and our friends for their infinite patience , encouragement, and

for understanding the long hours spent staring at the screens to make this vision a reality. This project is as much as yours as its ours. And also we thank Aarav Singh for helping us in the app too.

Table Of Contents

- 1.The spark of inspiration
- 2.Fix One Get 3 More Bugs
- 3.End Of BeatWave (Almost)
- 4.The First Beat
- 5.did it work?
- 6.Double Life: Students and Developers
- 7.V3: The New Era
- 8.May 7th 2026: The century mark
- 9.The Link Engine: 150 users
- 10.Liquid Glass The V4 Redesign

11.The Double Century: 200
Waves

12. The Flop Era

13. The Viral Thread

14. One Thousand : The
Promised Party

1

The Spark Of Inspiration

Every software project starts with a problem . For BeatWave that problem was a 2 minute unskippable advertisement. It was a evening of March 14th , 2026. My friend Akshansh Sinha and I

were on a voice call,
finally relaxing after
giving a stressful exam.
We were just trying to
unwind by listening to
some music together.
Spotify was the most
famous music app that
people really liked . as we
were vibing and zoning
out . A loud , irritating ,
unskippable commercial
blasted through our
speakers. We were
completely fed up with
this as we were annoyed
from it for the past few

months, but at the same time we refused to give money for spotify premium to stop ads.

In the heart of a moment a small joke slipped out.

“Lets make our own music app,” Akshansh said .

Right after those words left his mouth, the silence on the call was long and heavy. Neither

of us took it seriously at that moment. After a few minutes we hung on the call and went through the rest of our day normally , assuming it was just a typical evening chatter.

But Akshansh didn't let that idea slide .

Later in the night, my phone buzzed. It was a text from Akshansh :
“Brooo we found an open

source github repository named Echo music” we downloaded the app asap and started testing it, at that time Echo music had 500k or 600k downloads, we were amazed from the app.

After the tsting was done I texted Akshansh and said “ok so lets make our own app”

I was still a little skeptical about this decision because I always thought

that making an app is really tough .

The very next day on 15th march 2026 , we locked down the iconic name: BeatWave.

The app didn't always look as smooth as it looks today . in its very first version the app looked really raw and basic it was really laggy but it worked.

Getting the name and the
template was really easy
but the hard things were
about to begin.....

With the name *BeatWave* officially locked in, we felt completely unstoppable. We had the vision, we had the open-source base, and we had the restless energy of two friends convinced we were about to shake up the entire streaming scene. In our minds, the hardest part of the journey was already behind us. We had survived the brainstorming phase, found an idea that actually excited us, and claimed an

identity for our project. From where we were sitting, turning that idea into an actual working app on our phones seemed like a simple formality—just a few button clicks and a short wait.

"All we need to do is generate the APK, install it, and we're done," I told Akshansh over our voice call.

We genuinely believed the entire process would take five minutes. Maybe ten if the laptop decided to run slow. In our heads, the timeline

was already mapped out: we would hit compile, watch a quick progress bar slide across the screen, preparing the apk generation, after that we can spend the rest of the night streaming our favorite tracks on an app we could call our own. We were already laughing about how overcomplicated people made coding sound, half-expecting to be bragging about our effortless success before midnight, but god had some other plans.....

I stretched my fingers,
leaned back in my chair, and
confidently clicked **Build
APK**.

For a few seconds,
everything seemed to go
according to plan. The little
loading wheel spun, the
status bar inched forward,
and our excitement climbed
with every percentage point.
We sat on the call in eager
silence, just waiting for that
final green checkmark to
appear.

Then, the laptop fan suddenly spun up like a jet engine preparing for take off. The screen froze for a split second, and without any warning, the bottom console violently exploded into a terrifying wall of bright crimson text:

Plaintext

BUILD FAILED in 3 mins 42s

Execution failed for task
' :app:processDebugResources'.

> A failure occurred while
executing
`com.android.build.gradle.internal.tasks.Workers$ActionF`
`acade`

> Android resource linking
failed

The smug smiles on our
faces vanished into thin air.
The easy confidence we had
built up over the last two
hours evaporated in a single
second. There was no clean
installation prompt. There
was no shiny new BeatWave
icon popping up on our home

screens. There was only an endless, incomprehensible cascade of Gradle errors, broken paths, and missing dependencies that looked more like an ancient cryptographic puzzle than a music app.

The silence on the call returned, but this time it wasn't the silence of inspiration. It was the heavy, sinking realization that we were completely out of our depth. The happiness on our face slowly faded and our

heart had shattered into
millions of pieces before we
even got started, and
BeatWave was broken before
it was ever truly born.

2

The “Fix one get 3 Bugs”

If somebody had ever told me that coding was just typing random stuff and sipping coffee to stay awake, I would have laughed right in their face. By day four of trying to build BeatWave, it felt less like writing a simple program and more like actual

rocket science. We had severely underestimated what it took to make an app work, and that early arrogance was coming back to bite us hard.

When we started, we thought software development was a straight line: you download some open-source code, tweak a few settings, slap a custom logo on top, and magically end up with a polished app. In reality, it felt more like navigating an endless minefield

blindfolded. Every single time we thought we had scored a small victory, the app found a completely new way to humble us. We would spend three hours fixing a broken layout, finally watch it look halfway decent, and then the compiler would slam the brakes and throw another massive wall of cryptic Gradle errors right into the console.

It was like the code was actively fighting back against us at every turn.

What made the situation even more daunting was the sheer scale of the challenge ahead. We were only four days into what would turn out to be an exhausting, twenty-day marathon of non-stop trial and error. At that point, the dream of having a fully functional music player on our phones felt miles away—almost impossible, to be completely honest. We were trapped in a vicious loop that tested our patience every single night: tweak one minor line of code, fix a

single bug, and immediately watch three brand-new red error logs explode across the screen.

Our voice calls slowly shifted from excited planning sessions into hours of pure confusion. Akshansh and I would sit with our headsets on, staring at endless lines of dependencies, mismatched SDK versions, broken file paths, and build scripts until our eyes burned. We would try to decipher stack traces that looked like an ancient

alien language, frantically searching through obscure forum posts from eight years ago, only to find threads that ended with "never mind, I solved it" without any actual explanation.

There were moments when the silence on our call was deafening. We were tired, frustrated, and running on pure stubbornness. A project that had started as a casual joke over a voice call after an exam had spiraled into the most demanding and head-

scratching challenge either
of us had ever taken on—and
we were still only on the first
leg of the journey.....

3

End Of BeatWave (Almost)

By the time we reached the final stretch, our energy was running completely on fumes. For weeks, our entire day-to-day existence had devolved into a relentless, exhausting loop of build failures, broken dependencies, and glitchy

features. We had stared at endless red logs in the terminal for so long that the lines of code were literally beginning to blur together into a meaningless digital haze. The initial spark, the unbridled excitement, and the bold confidence we had shared on day one when we coined the name *BeatWave* were now hanging by an absolute thread.

That evening, Akshansh and I jumped onto a voice call for what we silently

acknowledged might be our last serious attempt. We were thoroughly burnt out. We had spent the entire afternoon untangling a stubborn, complex dependency conflict that had completely crippled the core audio service. It wasn't just a minor visual bug or an offset button; it was a fatal system-level crash that stopped the app from even opening on our test devices.

"Alright, I completely patched the Android

manifest, cleaned the cache, and restructured the audio player configuration from scratch," I said, my voice sounding flat and drained from sheer mental fatigue. "This is literally every single trick we know. Let's see what happens."

"Just run it," Akshansh said quietly. "Let's see."

I took a deep breath, hovered over the terminal, and hit the compile command one final time.

Neither of us said another word. The silence on the call grew heavy and suffocating as the laptop fans ramped up to maximum speed, whirring like an engine pushed to its absolute breaking point. We watched the terminal's progress bar slowly inch forward, line by line, second by second. Two minutes passed. Then five. Then eight.

Usually, our builds crashed within the first thirty seconds, so as the timer kept

ticking upward toward ten minutes without throwing an instant exception, a tiny, desperate flicker of hope began to build inside us. Maybe the long build time meant the system was finally linking all the binaries properly. Maybe the silence meant the compiler had finally accepted our fixes. We sat there, hands hovering near the keyboard, daring to believe that our weeks of sleepless troubleshooting were about to pay off.

Then, at the 10-minute mark, the screen abruptly flashed, the fans began to spin down, and the console delivered a crushing blow:

Plaintext

BUILD FAILED in 10m 18s

Execution failed for task
' :app:mergeDexRelease'.

> A failure occurred while
executing
com.android.build.gradle.internal.tasks.DexMergingTask

The shock hit us like a physical weight. We just sat

there, staring at the bright red text in complete and utter disbelief. We had checked every single semicolon, resolved every visible version mismatch, and followed every official guide step by step. Everything in our logic said that this was the one—that we were finally due for a clean, green BUILD SUCCESSFUL banner. Instead, the compiler had strung us along for ten agonizing minutes of false

hope only to slam the door shut right in our faces.

"Ten minutes..." Akshansh whispered, letting out a bitter, exhausted sigh through his mic. "Ten whole minutes just to throw another Dex error."

I leaned all the way back in my chair, staring up at the ceiling, feeling an overwhelming wave of defeat wash over me. We had made a quiet agreement before starting the build that this would be our final attempt.

We had completely run out of ideas, run out of forum threads to read, and, most importantly, run out of patience.

With heavy, hesitant hands, I moved the cursor over the main project folder. We were genuinely ready to close the editor, delete the local repository, shut down our laptops, and walk away from BeatWave forever. Was this the end of BeatWave?....

4

The first beat

Our fingers hovered in to the top left section of the software which we were using I was about hit “Delete” it would take less than a minute to wipe our whole 2 week progress, endless lines of codes and every single error logs.....

As I was about to hit “Delete” Akshansh Said “are you really about to delete our whole 2 weeks progress???”

I paused for a second rethinking my life choices but keeping them aside I replied “of course, we didn’t just try to fix everything we wasted all of our time trying to make the app work” I said , after a minute of silence Akshansh said “listen don’t delete the app right now just go get

some rest transfer the files to me and I will try to fix it one last time.”

I sent all the files to him and slammed my laptop shut. We literally forgot about it as our new session was about to start, so on the first day of our session when I asked Akshansh about the app he said “I had checked everything in that file and I saw some errors in it, I will fix it today it self and try to make apk for the one last time” I still didn’t believe him

because I had found every error in that code and told Akshansh what to fix but still it was giving an error. Around 4 o'clock when I reached home I texted Akshansh and said "are you making the app right now?" he replied with "yes hop on the call im trying" as I hopped on the call I saw him fixing some stuffs. After a few minutes he said, "everything is fix now the apk shall build now" we knew that it was our last attempt and we didn't have any hopes about our apk being

built because after I tried for 2 weeks the apk didn't work , it isn't possible that his apk attempt will , work right?... Anyways, when he started the apk build our hearts was beating so fast, as if someone was pounding on our chests. When the apk build got start we waited 5 minutes, 10 minutes, 20 minutes and even 30 minutes, As we were about to give up a messaged flashed into our screens. And guess what it was this message:-

BUILD SUCCESSFUL in 35m
04s

606 actionable tasks: 606
executed

"Wait..." Akshansh muttered,
his voice cracking slightly.

"Did it... did it actually
finish?"

"it built" I said with my voice
cracking. Then I quickly told
Akshansh to "go to the files
of the app and send me the
apk" Akshansh found the apk
"BeatWave-V1-release.apk"
he quickly sent the file to me
opened the file and clicked

“install app”. Loader was loading the apk could throw error at any moment..... but a message flashed and said “Apk installed”. The joy on our faces were not even imaginable we were so happy at that moment, when I went to the home screen of my phone I saw the good looking purple and blue logo with waves and the app name saying “BeatWave” we were feeling so proud and happy at that moment, it was unimaginable as I opened the app I saw the beautiful

user interface that me and Akshansh always dreamt of. Finally when we tried to play a song it threw an error “Syntax error, could not play song” we were so happy and so dissatisfied at the same time, but now we knew that it was possible to make BeatWave. So, we did some more research and found that it was an api issue that was not fixable from our side. But still we gave the files to Antigravity (an A.I assistant that we used for making our website) it said that the error

was fixed. But there was only one way to find out.... So with the new files we started to make the apk this time it didn't take so much of time as previous and a msg flashed into Akshansh's screen

BUILD SUCCESSFUL in 11m 04s

809 actionable tasks: 809 executed

This time it was not exciting because we knew that the apk will be generated for

sure, again I downloaded the
files and gave it a go

Lets see if a music plays in

3..... 2..... 1.....

and.....

5

Did It Work?

The wait was finally over, and the first song we picked to play on BeatWave was "505" by Arctic Monkeys. Back then, it was easily our favorite track. We used to keep it on repeat while chilling after classes, so it

made complete sense for it to be our test song.

I tapped on the track inside the app.

The loader icon in the center of the screen started spinning. My hands were literally shaking, and Akshansh was completely silent on our voice call, just listening in. We were crossing our fingers so hard, praying that another red error wouldn't pop up out of nowhere. Every single second felt like an hour.

Ten seconds passed. Fifteen seconds.

Then, right around the twenty second mark, the phone speaker clicked, and the warm organ chords of the intro started playing.

It was actually working. No crashes, no laggy audio, just pure music coming straight out of our own app. After two brutal weeks of non stop errors, broken builds, and moments where we were about to hit delete and quit, our hard work had finally

paid off. Just when we were ready to give up, Akshansh had stepped in and saved BeatWave.

"Bro, look at the lyrics tab," Akshansh said over the call.

I swiped up on the screen, and the synced lyrics loaded smoothly over the background. Everything was running without a single issue.

"Let's wrap up testing and release this to the public," I said.

At that point, we had only put a raw debug build on our GitHub. Because it was an unoptimized test file, it was quite laggy and heavy, but we still couldn't wrap our heads around the fact that two thirteen year olds had built a working music streaming app.

To show people what we built, I recorded a clean showcase video on my phone, showing the dark UI, the search bar, the player, and the synced lyrics. I

edited it and posted the reel on my Instagram.

We honestly thought only a handful of close friends would see it. Instead, the post blew up way beyond what we expected.

Within a couple of days, my inbox was flooded with over a hundred dm's. People from our mutual friends, and even random students from other schools were asking the exact same thing: where can we download this, and can

you send the APK link right now?

Since so many people were asking for it, we realized we couldn't just give them a buggy test build. We replied back to everyone and told them that the stable version of BeatWave would be dropping on the 5th of April, 2026.

We spent the next few afternoons right after getting back from school testing the app like crazy. We wanted to make sure the app wouldn't

crash when people searched songs or switched between Wi-Fi and mobile data. Our stable version was actually ready by the 4th of April, but we decided to take an extra day just to test every single button one more time.

Alongside the app fixes, I sat down and built our official website so users could easily download the APK without going through GitHub, and so we could keep track of how many people were downloading it.

On the 6th of April, 2026, we headed to school for our regular classes. Since phones are strictly not allowed inside school, we had to leave our devices at home. Walking through the gates and sitting through classes felt totally normal, but we spent every lunch break and free period talking about how many people might actually install it. Little did we know how much things were about to pick up once people got their hands on it.

Our download counter grew steadily, and by the 17th of April, 2026, we hit our first 50 downloads. It took a bit of time to reach that mark, but we did not lose hope at all. We kept reminding ourselves that real success takes time and patience, and that motivation kept us pushing for better updates.

Those fifty initial users helped us find small stutters and lag issues that we hadn't noticed before. We immediately got to work on

our next big release,
BeatWave Version 2.0. We
spent our afternoons
rewriting parts of the code to
make scrolling smooth,
cleaning up the caching, and
making the overall
experience fast and
lightweight.

On the 21st of April, 2026, we
officially launched Version
2.0. The lag that was there in
the original build was
completely gone, making it a
huge milestone for us.

Once v2.0 was live, we promoted it everywhere we possibly could. We shared the download link in all our friend groups, showed the app to our families, cousins, and relatives after school hours, and asked everyone to share it with their friends too.

Seeing the shocked reactions on people's faces when they realized that two thirteen year olds had made a fully working music app was the best feeling ever. BeatWave was no longer just

an idea on our laptops, it was out in the real world and people were actually using it.

The very next afternoon, things started getting even wilder. As soon as school ended and we walked out of the gates, a group of our classmates caught up with us on the road. Usually, conversations after school were about whatever homework had been assigned or how boring the last lecture was, but that day nobody was talking about

schoolwork. One of our friends pulled out his phone the moment we crossed the school boundary, opened BeatWave right in front of us, and showed us his favorite tracks already playing on loop. He told us he had deleted his other streaming apps because our player didn't push any advertisements down his throat. Hearing someone say that in person hit completely different than reading text messages on a screen.

By the time I reached home and kicked off my shoes, my phone was blowing up again with notifications. People had taken the download link from our website and were forwarding the APK file directly on WhatsApp. It was spreading from one group chat to another, reaching people we didn't even know. Cousins living in different cities were texting us asking if we really coded the whole thing ourselves, and relatives were calling our parents to talk about it. Our parents,

who used to get annoyed by the endless keyboard clicking and laptop fans spinning in our rooms, finally understood what we were trying to build all along. Seeing their proud smiles was something neither of us will ever forget.

Later that evening around 5 o'clock, Akshansh and I jumped back on our routine voice call to check the server statistics. When we opened the dashboard, our jaws literally dropped. The

counter that had taken weeks to hit 50 was now ticking up every couple of minutes. We sat there watching real people load up playlists, search for artists across different genres, and stream music directly through our infrastructure. It was insane to think that just a few weeks prior, we were staring at a red BUILD FAILED screen, ready to quit and erase every file from existence.

Of course, with more people using the app, our inboxes weren't just filled with compliments anymore. People began sending long lists of suggestions and feature requests. Some wanted a sleep timer so the music would turn off automatically when they went to bed. Others asked if we could add high-resolution audio streaming or the ability to import their existing playlists from other platforms with a single tap. Some users on older Android

devices even reported that the artwork took a couple of extra seconds to load when their internet connection was weak.

Instead of getting overwhelmed by all the feedback, it made us even hungrier to improve. We realized that releasing an app wasn't the finish line at all; it was just the start of an entirely new journey. We pulled out a blank notebook and started writing down every bug report and every

feature request that came in, categorizing them by priority.

We had gone from two kids complaining about annoying ads after an exam to running a full community project with active daily users. BeatWave was running out in the wild, and we were already planning our next move. This was only the beginning of BeatWave

6

Double Life: Students and Developers

The hardest part about working on BeatWave was that for eight hours every weekday, the project had to stop existing.

In our school, like basically every school in India, bringing a mobile phone was strictly forbidden. There were

random bag checks by the teachers at the front gate, and if someone got caught with a device, it meant an immediate call to their parents and having the phone locked inside the principal's cupboard for the rest of the term. You walked past the security guard, and your entire digital life went completely dark until the final dispersal bell.

To make things even more complicated, Akshansh and I

were not even in the same classroom.

When the new academic session started in April, the school reshuffled the batches. I ended up in section A, right on the ground floor, while Akshansh got assigned to section D ,also in the groundfloor we used to meet each other during lunch breaks. We didn't share a single lecture, our subject teachers were completely different,

That meant the only time we could actually coordinate anything during the entire school day was the twenty-minute lunch recess.

Every day when the 11:50 bell rang, the classrooms cleared out into total chaos. Hundreds of kids would rush to the crowd around the canteen counter to buy snacks and juice packs. While everyone else ran, Akshansh and I had our own fixed routine.

I would grab my tiffin box from my bag, and I would go to his class to take lunch there, sometimes he used to come on my class too, then Akshansh would show up holding his lunchbox in one hand and a folded rough notebook in the other.

"Did you check the server before leaving for the bus?" Akshansh would ask

"Yeah, forty-one new downloads came in after 9:00 PM," I told him, keeping an eye on the staircase.

"We never wrote a retry policy for network handover. The Android system drops the socket when the IP switches from the home router to the cell tower, and our ExoPlayer instance just gives up."

Without laptops, phones, or an internet connection in front of us, we had to build BeatWave on paper.

Akshansh flipped to the very last pages of his rough register. The front of that notebook was filled with

history dates, social science definitions, and English question-answers, but the back twelve pages were covered in hand-drawn app screens, database schemas, and scribbled lines of logic.

He rested the notebook on his knees and started sketching out the retry listener right next to a half-finished diagram of a water molecule.

"Look," he whispered, tapping the paper with the tip of his pen. "If we wrap the

streaming call inside a network callback function, we can check the connection status every two seconds. If the Wi-Fi drops, the app pauses the buffer pointer for three seconds instead of crashing, waits for the mobile data interface to handshake, and then resumes the stream from the exact same byte position."

I took the pen from his hand and looked at the logic.

"What if the mobile data signal is too weak? If

someone is traveling in an auto and the network drops to 2G, the buffer will choke anyway and freeze the main thread."

"Then we add a timeout," I said, scribbling a quick conditional statement underneath his lines on the lined paper. "If the network doesn't respond within five seconds, the player shows a small snackbar alert at the bottom saying 'Poor network' instead of killing the

background playback service."

We spent fifteen minutes hunched over those notebook pages, crossing out broken logic, rewriting variable names, and drawing arrows between different blocks of code. We were talking so fast and writing so frantically that our food barely got touched.

"Meet at the court yard at dispersal," I whispered as we reached the stairs. "I'll test

the socket retry code the second I get home."

He gave me a quick nod, ran up the stairs toward section D on the third floor, and I hurried down the corridor to section A just as the warning bell for the next period rang.

For the rest of the afternoon, paying attention in class was nearly impossible. While my English teacher was explaining a poem line by line, my eyes were glued to the wall clock hanging above the front door. The red

second hand seemed to move in slow motion. I had the retry logic burned into my brain, and having to sit still through two more hours of literature and geography while a live bug was breaking playback for real users felt completely frustrating. Every time the teacher turned to the blackboard, I would jot down another reminder in the margin of my notebook: *check ExoPlayer socket timeout, clean memory heap on resume, update website link.*

When the final bell finally rang at 2:55 PM, the sound was like an alarm going off.

The classrooms emptied out instantly. I threw my notebooks into my bag, pulled the zipper shut, slung the strap over my shoulder, and joined the flood of students moving toward the school gates. I met Akshansh near the courtyard.

"Everything ready?" he asked over the loud chatter of hundreds of kids leaving.

"Notebook is in my bag. I'll boot the laptop as soon as I drop my shoes," I said. "Call me at five."

"Deal."

I took my regular transport back home. The 1 hour ride through the afternoon heat, dodging traffic, potholes, and crowded road crossings, felt longer than the entire school day. While other kids in the vehicle were joking around, sharing snacks, and talking about cartoon shows, my mind was running through

terminal commands, Gradle build files, and user feedback emails.

The minute I reached home, the school half of my day was officially shut off.

I took off my uniform, splashed cold water on my face, and ate whatever lunch my mother had kept on the table as fast as I could. By ten minutes to five, I was inside my room with the door pulled shut, my laptop powered on, and Android

Studio loading up on the screen.

At 5:00 PM sharp, my phone buzzed on the desk.

It was a Discord notification from Akshansh:

"Voice call started. Hop on."

I clicked join, put on my wired headset, and heard him immediately: "I've already imported the lines we wrote on the stairs into the local test branch. Let's see if the socket handler actually compiles."

For the next hours, our rooms became our workspace. We weren't eighth-graders sitting in separate classrooms anymore; we were two developers maintaining an app.

Akshansh shared his screen, showing the code we had written by hand during lunch. It had looked clean on paper, but Android compilers don't care about handwriting. The second he hit compile on the test module, the console spat out two syntax warnings

and an unhandled null-pointer exception on line 142.

"The callback isn't checking if the context is null when the app goes into the background," I pointed out, watching his screen share. "If the user switches from Wi-Fi to data while their phone is locked, the context gets destroyed, and the socket retry throws a fatal crash instead of waiting."

"Good catch," Akshansh said, typing rapidly on his

keyboard. "I'll pass the application context instead of the activity context. That way the background audio service holds onto the network reference even if the screen turns off."

He saved the file, cleaned the build cache, and pushed the patch to my machine. I pulled the commit, and flashed the debug build straight onto my device.

I opened the updated BeatWave build, searched for a song, and hit play.

While the track was streaming smoothly over my home Wi-Fi, I walked across my room, pulled down the quick settings menu on my phone, and turned off the Wi-Fi toggle to force it onto mobile data.

For a second, the audio paused. The buffer indicator spun.

Neither of us said a word on the call. We watched the terminal logcat output scroll by, waiting for the dreaded

red FATAL EXCEPTION crash log to show up.

Two seconds passed. Then the console printed a clean message:

Plaintext

Network changed:

TRANSPORT_CELLULAR.

Resuming playback at 01:14.

The phone speaker kicked back in instantly, picking up the song at the exact millisecond where it had paused, without a single stutter.

"Bro, it worked!" I yelled into the mic. "The handover logic we wrote on the stairs actually held the connection!"

"No way!" Akshansh laughed over the call. "The retry loop held?"

"Clean resume. No crash, no memory leak, nothing."

Right at that exact second, before we could even push the fix to GitHub, a loud knock rattled my bedroom door.

My mother pushed the door open, looking at my headset, the open code windows, and the messy desk.

"Aarav," she said firmly. "It's 7:00 PM. Tuitions start in twenty minutes, and you haven't even revised from this morning. Close the computer right now."

I let out a slow sigh, looking at the terminal screen where the green success logs were still sitting. "Yes, Mom. Just shutting it down."

I leaned into the mic:

"Akshansh, you heard that?"

"Yeah," he sighed. "My mom just came into my room too with my math homework. I have to finish the algebra exercises before dinner or I'm dead."

"I'm pushing the commit right now," I said, typing `git push origin main` into the terminal. "The network fix is live. We'll check the crash reports tomorrow during lunch."

"Done. See you at lunch
break tomorrow ."

"See you, bro."

I shut down the laptop,
watched the screen go pitch
black, and took off the
headset.

Within thirty seconds, the
developer had to disappear. I
pulled my school bag off the
floor, took out my maths
register, opened my pencil
box, and sat down at the
dining table under the bright
white tube light to solve the

exercises until dinner was served.

Sometimes, keeping up with that split routine was completely exhausting. Other students had the luxury of finishing their school day, coming home, watching videos, and hanging out with their friends without a single worry in their heads. Akshansh and I were dealing with school rules, separated classrooms, and strict parents on one side, while trying to run and

maintain a real, open-source streaming app on the other.

We had to design algorithms on paper, hide code inside subject notebooks, and squeeze an entire day's worth of development into a couple of tight afternoon hours before our homework started.

Yet, sitting there with my textbooks open, listening to BeatWave quietly playing through a pair of wired earphones on my desk, I knew neither of us would

have changed a single thing.
We weren't just two kids
stuck in section A and
section D waiting for the final
bell. We were building
something real, one line of
code at a time.

7

V₃: The new era

By the end of April, BeatWave was no longer a secret experiment shared between two friends. It was active, it was out in the real world, and people were actually using it every day. Version 2.0 had fixed the worst of our initial

lag and stopped the app from crashing every time someone walked out of range of their home Wi-Fi, but as the user numbers climbed, our expectations started changing too.

Version 1.0 had been about survival—just proving that we could compile an APK that made sound come out of a speaker. Version 2.0 had been about damage control, rushing out patches so our classmates wouldn't roast us when the app froze on their

devices. But with Version 3.0, Akshansh and I wanted something completely different. We didn't just want an app that worked; we wanted an app that looked and felt like it belonged alongside the biggest streaming platforms in the world.

We wanted a complete overhaul.

The decision started, like everything else with BeatWave, during a hurried conversation.

"Look at the player screen on v2," Akshansh said, balancing his lunchbox on his knees while pulling out our battered rough register.

"It works, but it still looks like an open-source template.

The corners are too sharp, the font doesn't match the album art, and whenever a song has a long title, the text just clips through the edge of the layout instead of marquee-scrolling."

"And the queue management is completely rigid," I added,

chewing quickly on my lunch while watching the corridor for prefects. "If someone wants to reorder their songs, they literally can't. You tap a track, and you're stuck with whatever order the search query gave you. People are messaging us on the website asking for a proper drag-and-drop queue, an equalizer, and offline caching."

"Offline caching is a massive rewrite," Akshansh warned, his brow furrowing as he tapped his pen against the

lined paper. "Right now, the ExoPlayer instance streams directly from the remote URL. If we want local caching, we have to build a local database layer, manage file encryption so downloaded chunks don't get corrupted, and store metadata like high-res artwork without eating up all the phone's internal storage."

"Then we rewrite it," I said.

"We can't keep calling it an upgrade if all we do is change the accent colors. If we're

doing Version 3.0, we rebuild the player from the inside out."

Akshansh looked at me for a second, then a grin broke across his face. He flipped to a clean page at the back of the notebook, drew a bold box across the top, and wrote: *BEATWAVE v3.0 ARCHITECTURE*.

For the next two weeks, our afternoons followed a relentless, highly organized grind. Every day at 2:55 PM, the final school bell would

ring, and we would rush out of our separate classrooms—me sprinting down from section A, Akshansh hurrying down from section D on the third. We met by the corridor just long enough to exchange quick updates before heading home on our separate routes.

By 5:00 PM sharp, without fail, my phone screen lit up on my wooden desk:

"Call started. Let's build."

I put on my headset, clicked into our voice channel, and brought up Android Studio. The fan on my laptop immediately spun up, humming its familiar low buzz as the project indexed.

The first major task was the visual identity. Version 2.0 had stuck to basic dark mode—mostly harsh blacks and flat grays. For v3.0, we wanted a signature look. We spent hours testing subtle color hex codes, settling on a deep midnight-slate palette

blended with vibrant purple and cyan accents that matched the BeatWave logo.

I took charge of the user interface redesign. I replaced the clunky stock Android sliders with custom-drawn waveform seekbars that pulsed smoothly with the audio. Instead of static text boxes, we coded dynamic marquee text views that gently scrolled track names across the screen if they were too long to fit. We also implemented a dynamic

background engine:
whenever a song started
playing, the app extracted
the dominant colors directly
from the album art and
painted a soft, blurred
gradient across the entire
playback screen.

"Look at this render," I said,
sharing my screen with
Akshansh over the call. I
loaded up a track with bright
blue and neon pink album
artwork. The screen instantly
adapted, the dark interface

melting into a moody,
glowing ambient backdrop.

"That looks incredible,"
Akshansh admitted, leaning
closer to his mic. "It doesn't
look like a student project
anymore. It looks premium."

"Now you just have to make
sure your new audio pipeline
doesn't choke when this
runs," I joked.

"Don't worry about the
pipeline," he fired back.
"Watch this."

Akshansh had taken on the hardest technical challenge of v3.0: the offline caching and queue management engine. Up until then, if a user listened to their favorite song ten times in an afternoon, BeatWave downloaded that exact same audio data ten separate times from the remote server, burning through mobile data and adding annoying buffer delays every single time.

To fix this, Akshansh spent days implementing a two-tier cache structure using Android's SimpleCache combined with a local SQLite database. Whenever a track began streaming, the application quietly mirrored the incoming byte stream into a temporary encrypted storage folder on the phone's internal memory. The next time the user tapped that same song, the app bypassed the internet entirely, loading the audio

instantly from local storage
in less than half a second.

"Check the log," Akshansh
said, pushing his latest
branch to our shared
repository.

I pulled the code, and
compiled the debug build. I
searched for a track, let it
play for thirty seconds,
Instant sound. Zero
buffering.

The progress bar began
moving immediately, the
cached album art loaded in

full resolution, and the terminal output printed a clean, triumphant log:

Plaintext

MediaSource:

LOCAL_DISK_CACHE hit.

Stream initialized in 112ms.

"It worked!" I shouted into the headset. "Zero delay!"

"Instant playback," Akshansh said, laughing with pure relief.

Building v3.0 wasn't just smooth progress, though. Every time we pushed the

application's features forward, we ran straight into the harsh limits of budget Android hardware.

One afternoon in mid-May, flawlessly—silky smooth transitions, vibrant dynamic backgrounds, and zero memory spikes.

By the starting of May, BeatWave v3.0 was finally nearing completion. We had redesigned the entire interface, implemented local caching, built a real drag-and-drop queue, added

custom playback speeds,
and optimized the memory
footprint to run smoothly
even on five-year-old
hardware.

The final test came on a
warm Friday evening. We had
set an internal deadline to
build the release binary
before 6:00 PM, right before
our evening tuition classes
started.

"Manifest checked," I said,
running through the pre-
compilation checklist on our
call. "Version code bumped

to 3.0. Proguard rules updated. Build cache wiped clean."

"Trigger the release build," Akshansh said quietly. "Let's see what we've got."

I hovered my mouse over the terminal, typed `./gradlew assembleRelease`, and pressed enter.

The fans kicked in. The laptop purred with steady, rhythmic energy. Unlike the chaotic days of Version 1.0, when every build felt like gambling with fate, we knew

our code now. We had spent weeks understanding every library, every dependency conflict, and every background permission.

Four minutes and twelve seconds later, the console printed the words we were waiting for:

Plaintext

BUILD SUCCESSFUL in 4m
12s

428 actionable tasks: 428
executed

I exported the signed APK:
BeatWave-v3.0-
Universal.apk.

I installed it directly onto my phone, opened the app, and plugged in my headphones. The new deep-slate interface opened with a fluid, seamless animation. The search bar responded instantly to keystrokes without lag. The song loaded in a fraction of a second, the waveform pulsed smoothly across the bottom of the screen, and the background

subtly glowed with the exact colors of the cover art.

I held up my phone, switched on screen sharing, and showed Akshansh the finished product running in real-time.

"We actually did it," he said, his voice quiet with genuine pride. "Look at that. That doesn't look like two eighth-graders built it in their bedrooms after school. That looks like a real product."

"It is a real product," I replied, looking at the clean logo sitting on my phone's screen.

Right at that moment, my bedroom door creaked open, and my mother stood in the doorway, pointing at the wall clock. "Aarav, 7:00 PM. Coaching time. Shut the laptop."

"Shutting down right now, Mom," I smiled, completely unfazed this time.

I gave Akshansh a quick sign-off: "Branch is merged. Uploading the APK to the

website server right now. We announce v3.0 tomorrow morning."

"See you at lunch break."

"See you."

I pushed the release live to our web server, shut down the laptop, and packed my school registers into my bag. Version 3.0 was done. We had moved past the raw prototype phase, pushed through the hardware limitations, and built something we were truly proud of. BeatWave was

ready for the wider world. Did this update made us reach the first milestone of our lives?..

8

May 7th, 2026: The Century Mark

May 7th was the first day since we started this project where we did not open a code editor, touch a file path, or run a single test command. We were completely sick of looking at code. For weeks, our heads had been stuffed with

background services, network timeouts, and asset compression. What we cared about that day wasn't how the app was put together under the hood. We only cared about one cold, hard number: eighty-nine.

Eighty-nine was where our download counter had been stuck since the night before.

To anyone else, eighty-nine might sound like a decent start for two eighth-graders. But when you have poured every free afternoon of your

life into an idea, eighty-nine felt like being stranded on an island ten meters away from the shore. Our goal from the very beginning had been one hundred. It was the imaginary line between a silly school experiment that nobody took seriously and a real, living project that had genuinely reached people.

Before heading out the front door for school that morning, I refreshed the website on my phone one last time. Eighty-nine. Not a single person had

clicked download while we were asleep.

The problem with being a student in India is that once you cross the school gate, your connection to the outside world is severed completely. Phones are strictly forbidden. You walk through the entrance, show your school ID card, and hand over your daily life to bells, whiteboards, and classroom rules. Whatever was happening to BeatWave out on the web, we were

blind to it for the next seven hours.

Being in different sections made the silence even worse. I sat near the second-row bench in Section A, while Akshansh was stuck all the in Section D. We couldn't even exchange a glance across the room to ask if the other had heard anything before leaving home.

During our English literature lecture, the teacher was reading through a chapter about a boy who cant

remember his mother, but I couldn't process a single word. My mind was running through the people we had talked to. Had my cousin actually shared the website link in his college WhatsApp group like he promised? Did the older kids from my neighborhood ever click the install prompt, or did they just nod along to be polite and scroll past? When you have no way to check, your brain fills the quiet with worst-case scenarios.

The 11:50 recess bell felt like an escape siren.

I grabbed my tiffin box, ducked out of the Section A doorway, and cut straight through the crowd of younger kids sprinting toward the canteen.

A few seconds later, Akshansh came jogging down the stairs, his lunchbox tucked under his arm.

He didn't even say hello. He just sat down.

"Still eighty-nine when you left?" he asked.

"Eighty-nine on the dot," I replied, sitting beside him and popping open the metal lid of my tiffin. "Nothing moved all morning."

"Same here," Akshansh said, staring out through the railing at the dusty courtyard where students were . "Two guys from my bus said they would download it after tuition today, but that's hours away. We need eleven downloads. Eleven sounds so small, but

it feels completely impossible right now."

"What hurts is the kids in our classes who keep making remarks," I said, chewing quietly. "A boy in Section A saw the logo on my rough register yesterday and asked me why we were wasting our time when Spotify already exists. He told me nobody in our school is going to switch to an app made by students."

Akshansh gripped his lunchbox tighter. "That's why one hundred matters. Once

we cross into triple digits, it's not a joke anymore. You can laugh at a project that has ten downloads. You can't laugh when a hundred different people actively went out of their way to install it."

We spent the entire twenty minutes debating whether our website was clear enough, whether the download button was too hard to spot on small phone screens, and whether people were getting confused by the

installation prompt. Not once did we talk about code. We talked about human psychology—why people hesitate to try something new, and what it would take for ordinary listeners to give two unknown kids a chance.

When the warning bell rang, we packed our half-eaten lunches and split up, returning to our separate classrooms for the afternoon stretch.

The final two periods dragged on forever. Every sweep of

the red second hand on the classroom clock seemed slower than the last. By the time the final 2:55 bell sounded, the noise of benches scraping and bags zipping filled the room. I was out the door before the teacher had even capped her whiteboard marker.

I didn't stop to talk to anyone near the court yard. I stayed in the line and got into my bus with my sister and spent the entire journey staring at the road, watching

motorbikes and auto-rickshaws battle through the afternoon traffic. The heat outside was burning, but my hands felt cold with nervous anticipation.

I walked into my house at 4:05, dropped my bag onto the sofa, took off my school shoes, and ate whatever was on the kitchen table without tasting any of it.

By ten to four, my bedroom door was shut, and my laptop screen was glowing in front of me.

At 5:00 PM sharp, my phone lit up on the desk.

A message from Akshansh:
"Discord voice. Join right now."

I put on my headset, clicked into the call, and opened the browser tab where our real-time download logs were displayed.

The page finished loading.

The large number in the middle of the screen had moved.

93.

"Four downloads!" I shouted into the microphone.

"Akshansh, four people got it while we were sitting in class!"

"We're at ninety-three!" he yelled back, his voice jumping an octave. "Seven left! Just seven!"

For the next forty minutes, neither of us could stay in our chairs. We left our IDEs closed. We didn't care about bugs, layout adjustments, or updates. Our entire world had shrunk down to a single

webpage that refreshed its database query every sixty seconds.

Akshansh was pacing across his bedroom floor. I could hear his footsteps and the squeak of his door every time he turned around.

"Just think about it," Akshansh said over the line, his voice full of genuine wonder. "Right now, ninety-three people have our icon sitting on their home screens. When they plug in their earphones on the bus

or while doing their homework, they tap something we made."

"It doesn't feel real until it's three digits," I answered, keeping my eyes glued to the counter. "Seven people. Come on."

At 5:18 PM, the page reloaded with a faint flicker.

95.

"Ninety-five!" I yelled. "Five to go!"

"Five people!" Akshansh shouted. "Bro, we're five

people away from the century mark!"

"Don't touch anything," Akshansh pleaded. "Don't refresh manually. Don't touch the server. Just let the traffic do its thing."

Those next twenty minutes were pure, unfiltered suspense. The afternoon sunlight was slanting across my desk, lighting up the dust floating in the room. The only sound was the steady whir of my laptop fan and our own nervous breathing over the

headset. Every single minute felt like waiting for exam results to be posted on a school notice board.

At 5:38 PM, the counter updated again.

98.

"NINETY-EIGHT!" Akshansh screamed, his chair scraping violently across his floor as he stood up. "TWO! ONLY TWO LEFT!"

"BRO, TWO MORE DOWNLOADS!" I yelled back, standing up from my chair

with my headset cable
pulling tight. "TWO PEOPLE
IN THE ENTIRE WORLD JUST
NEED TO CLICK THE
BUTTON!"

"Should I send the link to my
cousin again?" he asked
frantically. "He's online right
now!"

"No!" I shouted. "We
promised we wouldn't fake it!
If we inflate the number
ourselves, the milestone is a
lie. It has to be real users
who actually want the app!"

"You're right, you're right," he said, breathing heavily. "No faking. Organic only."

We stood by our desks in absolute, breathless silence.

The clock on my taskbar turned from 5:41 to 5:42 PM.

The browser reached the end of its sixty-second auto-refresh cycle. The tiny blue progress bar zipped across the top of the tab. The numbers in the center box faded out for half a second while the database tallied the latest record.

Then, the new number
clicked into place.

100.

A clean, solid, three-digit
number.

For three whole seconds, the
call was dead silent. We just
stared at those three digits,
making sure the screen
wasn't glitching or showing a
false cache value.

Then the call completely
exploded.

"ONE HUNDRED!" I roared at
the top of my lungs,

slamming both hands down onto my wooden desk. "WE HIT A HUNDRED! IT'S ACTUALLY A HUNDRED!"

"LET'S GOOOOO!" Akshansh screamed back, laughing so hard he was almost coughing into his mic. "WE DID IT! ONE HUNDRED DOWNLOADS! THREE DIGITS, BRO!"

The rush that went through me was unlike anything I had ever felt before. It wasn't about the code, and it wasn't about technology. It was about validation.

One hundred real human beings had looked at an app created by two thirteen-year-old schoolboys, downloaded the file onto their personal devices, and trusted it enough to keep it there. Every single doubt, every sarcastic comment from our classmates, every afternoon spent sacrificing our free time, and every single moment we had felt like giving up completely vanished under the weight of that one number.

Akshansh was still chuckling on the other end, taking deep breaths.

"Your mom came in too?" he asked.

We sat there on the call for a long time, not doing anything at all. We just stared at the number 100 resting quietly in the center of the dark-mode dashboard. The classmates who said our app was useless, the frustrating days of being stuck at eighty-five, the struggle of having our phones locked away during

school hours—none of it bothered us anymore.

May 7th, 2026, had started as just another ordinary, exhausting school day. But by 5:42 PM, it became the day BeatWave proved it wasn't just a fantasy between two students. It was real, it was alive, and a hundred people were listening to the music we brought them. Now the next target was 200 downloads.

9

The Link Engine: 150 users

Crossing one hundred downloads changed the way Akshansh and I looked at BeatWave. Before May 7th, every update felt like an attempt to prove that we were not wasting our time. We were constantly trying to shut down the doubts from

kids in our grade who thought building an app was something only adults in tech companies could pull off. But once the counter touched triple digits, the insecurity faded away. We did not need to prove the app was real anymore. The number sitting on our site proved that for us.

Now, a completely new kind of pressure took over. When you have one hundred people using something you made, you cannot simply

abandon it or leave bugs sitting around. People expect the app to work, and more importantly, they expect it to get better.

Over the weekend right after the milestone, our Discord server and our Instagram page, @_beat_labs,(at that time, now our dev team has been renamed to vortex apps) started seeing more activity. We had set up our site at beatwave.oneapp.dev so anyone could easily get the APK and join our small

community, but our direct messages were quickly filling up with suggestions.

Most of the messages were positive, with people saying they loved the dark aesthetic and the lack of commercial interruptions. But one specific request kept showing up in our inbox over and over again from different users.

People did not just want to search for standard studio tracks by typing out song titles. They wanted

unreleased music, indie live sessions, pitch-shifted versions, instrumental slow beats, and custom remixes that you could only find floating around on YouTube.

One user sent a detailed message on Discord: *"Hey, the search bar works fine for popular songs, but whenever I try searching for a specific underground remix or a live performance, the results never show the exact video I want. Can you add an option*

where I can just paste the link directly?"

That single message sparked the idea for Version 3.2.

On Monday afternoon, the moment 5:00 PM rolled around, I put on my headset and joined our regular Discord voice channel. Akshansh was already online, his screen shared with an open text document where we kept our patch ideas.

"Did you see the feedback on the Instagram page?" I asked him through the mic.

"Yeah, people want to paste YouTube links directly," Akshansh replied. "It makes complete sense. The default search query only returns high-relevance matches based on metadata. If someone wants a specific two-hour study mix or a concert version that has an obscure title, the regular search algorithm pushes it

down to page four where nobody looks."

"How hard is it to implement?" I asked.

"In theory, it's simple," Akshansh explained, pulling up our core search handler.

"We need to intercept the text input inside the search bar. If the string begins with standard URL patterns like [https://youtube.com/watch] (https://youtube.com/watch) or

https://youtu.be/, the app should

bypass the general search API entirely. Instead, it needs to extract the unique video ID from the end of the URL, send a direct endpoint request for that specific stream, and hand the audio over to the player."

It sounded straightforward on paper, but writing the code was a different beast altogether.

We gave our files to antigravity hoping it will make the update for us.

Our first big roadblock came from how messy URLs can actually be. When people copy a link from their phone's share sheet, the link rarely comes as a simple, clean address. It often includes tracking tags, timestamp parameters like `&t=45s`, playlist identifiers, and channel tags.

On our first test run, I grabbed a link from a live acoustic performance and pasted it into our test build's search bar. The input field

took the link, the screen froze for three seconds, and then the app threw an unhandled string index exception and closed itself.

"It choked on the query parameter," I told Akshansh, looking at the terminal logs.

"The share sheet link had ?si= attached at the end for tracking. The string trimmer didn't account for anything after the video ID, so it tried to parse the tracking parameters as part of the media ID itself."

"Right, we need a bulletproof regex cleaner," Akshansh said, typing a prompt to antigravity. "It has to strip away everything—timestamps, channel referrals, playlist loops—and isolate only the eleven-character video key. No matter what kind of garbage tracking link the user pastes in, the app must sanitize it before hitting the network."

We spent that entire evening testing every conceivable URL format. We tested

standard desktop links, mobile shortlinks, embedded links, and URLs with timestamps. Akshansh refined the parsing script until the function could take any raw string from a clipboard, extract the exact eleven characters, and trigger the player in under half a second.

While he finalized the link engine, I went through the entire application to tackle another major complaint from our users: visual

sluggishness during screen transitions.

In Version 3.1, while we had reduced the file size drastically, the app still had occasional micro-stutters when switching from the search screen to the now-playing view. The album artwork transition was slightly choppy on budget processors because the layout was reloading the entire view hierarchy instead of keeping it in memory.

I spent hours fine-tuning the view model and adjusting the animation curves. I replaced the heavy stock layout animations with lightweight hardware-accelerated transitions. Now, when a user tapped a song or pasted a link, the album art floated into view seamlessly while the background colors shifted softly without dropping a single frame. It was under-the-hood tuning, but it made the entire user experience feel significantly

more polished and responsive.

By Wednesday, May 13th, the build was rock solid. We spent an hour testing the search bar with every strange, obscure remix link we could find on the web.

Every single one loaded cleanly. You could copy a link from a browser, open BeatWave, tap paste, and the music started streaming almost instantly.

"This is ready," Akshansh said with satisfaction. "It feels ten times faster than v3.1."

We prepared our release notes to post across our socials and our website:

Plaintext



BeatWave v3.2 Release Notes

We are back with another update to make your BeatWave experience smoother and more powerful! In this release, we're introducing a highly

requested feature that changes how you find and play music, along with some under-the-hood tuning.



What's New?

Search by YouTube Link:
Finding your favorite tracks just got infinitely easier. Instead of typing out long titles, you can now paste any YouTube URL directly into the search bar to instantly pull up the audio. Perfect for deep cuts, remixes, and live performances!



Bug Fixes & Improvements

Smoother Performance:
Optimized animations and transitions to make navigating the app feel much more fluid and responsive.

Under-the-Hood Tuning:
Squashed several minor bugs and performance hiccups to keep the app running fast and stable.



Connect With Us

Stay updated, report bugs, or just hang out with the community:

Website:

beatwave.oneapp.dev

Discord: Join our Server

Instagram: @_beat_labs

I compiled the final production binary, updated the release assets on GitHub, and uploaded the new APK file to beatwave.oneapp.dev. We posted the release announcement on our

Instagram and dropped the update ping in our Discord server.

At that moment, our total download counter was sitting at 108.

We had set an internal target for ourselves: reach 150 downloads with Version 3.2. Crossing from 100 to 150 seemed like a realistic step, but we knew that reaching people beyond our initial circle would depend entirely on whether users found this update genuinely useful.

The response was faster than we anticipated.

Within two hours of posting the update, our Discord notification bell started ringing. Users were dropping messages in the general chat:

"The YouTube link search is insane! I can finally listen to live sets without opening a browser."

"Animations feel super smooth on my phone now. Good job guys."

Hearing that feedback gave us a massive wave of momentum. People began sharing the update across their own social circles. On our dashboard, the download counter began moving at a steady pace. It did not explode all at once, but every single time we checked, another two or three downloads had registered.

By Thursday evening, the counter had climbed to 124.

When I got home that afternoon at 4:05 PM, dropped my bag, and logged into my laptop, the dashboard was updating with numbers we had never seen before.

At 4:30 PM, the counter hit 138.

At 5:00 PM, when Akshansh joined our Discord call, the number climbed to 143.

"Look at the traffic chart," Akshansh said, his voice buzzing with excitement. "We

have twenty active sessions on the site right now!"

We sat by our desks, watching the live traffic logs scroll by. It felt different from the tense, nail-biting silence of May 7th when we waited for download number 100.

This time, we were not stressed or desperate to prove ourselves. We were just excited to watch the community grow around something we had built from scratch.

At 5:28 PM, the counter jumped: 147.

Five minutes later, two more clicks registered: 149.

Then, at exactly 5:36 PM, the page refreshed automatically, and the counter flipped to **151**.

"We crossed it!" I laughed into the mic. "One hundred and fifty downloads!"

"A hundred and fifty in less than two weeks after hitting a hundred," Akshansh said

proudly. "The link search completely did its job."

Reaching 150 downloads proved to us that BeatWave was not just a one-time novelty that people installed once and forgot about. It was evolving into a genuine platform with a real community behind it.

Through Version 3.2, we learned that listening directly to our users and refining small details was just as important as writing complex features.

We had started this project out of pure frustration with ads, but now, sitting at 151 downloads with a working platform and an active community, the finish line seemed far away. We updated the site, saved our logs, and began talking about what would come next. BeatWave was moving forward, and we were ready for whatever was coming.

10

Liquid Glass: The v4 Redesign

Crossing 150 downloads gave us a brief moment to celebrate, but as the days went by, Akshansh and I realized that BeatWave was running straight into an invisible ceiling. Version 3.2 was fast, and the YouTube URL engine worked like a

charm, but under the hood, the app was still carrying old technical baggage. Up until that point, parts of the application were still relying on older structures, patchwork fixes, and layout styles that had been carried over from our earliest prototypes. The app looked decent, but we wanted something that felt distinct, modern, and completely ours. We did not just want flat buttons and basic dark gray surfaces anymore. We wanted a style that had

depth, translucency, and personality.

That desire sparked the concept of our Liquid Glass aesthetic, and with it, a total redesign of our visual identity that became BeatWave Version 4.0.

The conversation started on an afternoon right after 5:00 PM when we hopped on our routine Discord call.

Akshansh was already screen sharing his development environment,

looking through our layout folders and XML templates.

"If we want people to take BeatWave seriously as a modern audio player, we need to completely rethink the visual identity," Akshansh said, pulling up a mockup he had been sketching. "The sharp card edges and solid dark slabs look like an open-source utility from five years ago. Look at modern design languages. Everything uses soft gradients, frosted overlays, and fluid depth."

"Like liquid glass," I said, looking at the way light filtered through the translucent layers on his screen. "Translucent floating panels where the blurred artwork peeks through from the background, subtle glowing borders, and dynamic UI elements that respond to touches without cluttering the screen."

"Exactly," Akshansh agreed. "And if we're overhauling the visual language to this Liquid Glass style, the old logo has

to go too. The original icon was decent for our early builds, but it's way too childish. It looks like a rough draft. We need a sleek, redesigned wave mark that matches this new glass aesthetic."

The logo redesign became our first major priority. We spent hours testing sketches, playing with vector paths, and trying out different gradients. Our old logo had been a basic, rigid waveform that felt flat and

lifeless. For the new look, we wanted something that represented both liquid movement and sound waves.

I took the lead on the vector canvas, experimenting with smooth curves and aerodynamic sweeps. We blended a deep neon cyan into a vibrant electric purple, shaping the wave so it felt like it was rippling right off the glass surface. We spent countless hours tweaking anchor points, making sure the curves felt balanced and

organic rather than stiff. When we added a subtle outer glow and exported the high-resolution vector, the difference was night and day. It looked clean, futuristic, and premium. Whether it was sitting as a tiny icon on a phone's home screen or placed boldly at the top of our landing page, it finally felt like a real brand.

Next came the actual implementation of the Liquid Glass interface inside the app itself.

Designing frosted glass effects in Android is notoriously tricky because rendering live blurs in real time can quickly overheat budget processors and drop frame rates. We could not just slap heavy blur filters everywhere and hope for the best. If an app chokes while scrolling through a playlist, users will immediately uninstall it, no matter how pretty it looks.

Instead, we carefully crafted lightweight semi-transparent

overlays, soft layered gradients, and delicate borders that caught the light around cards, playback controls, and menus. We used subtle alpha values, carefully chosen hex codes, and layered backgrounds to create an illusion of depth without forcing the GPU to calculate intense Gaussian blurs on every single frame. When a song played, the album artwork dissolved into a soft ambient backdrop behind the frosted panels, giving the entire screen a

genuine, translucent liquid glass feel.

To give users even more control over this new aesthetic, we built an entire Custom Themes engine. We did not want listeners to be locked into our default midnight tones. With the new theme picker, anyone could personalize BeatWave with an entirely fresh palette of colors that matched their own style and energy, all while keeping that sleek frosted glass depth intact.

We spent late afternoons experimenting with color contrast ratios to ensure that whether a user picked an emerald tone, an electric violet, or a fiery crimson, the text remained perfectly legible and the glass borders popped cleanly against the background.

We did not just want to put a fresh coat of paint on a cracked foundation, though. If we were going to build this modern Liquid Glass

interface, we had to upgrade the engine beneath it.

"We can't run this new visual engine on top of outdated architecture," I pointed out.

"If we put layered transparency and smooth animations on our old media pipeline, it will stutter on older devices. We need to modernize the whole core stack."

That turned Version 4.0 into our most ambitious engineering push yet.

First, we fully transitioned our application logic to modern Kotlin, taking advantage of asynchronous coroutines and cleaner memory handling to prevent unexpected crashes and memory leaks. In the earlier versions, managing background tasks in Java often felt messy, requiring callback chains that made debugging a headache. With Kotlin flows and coroutines, we could handle network requests, stream fetching, and interface updates

asynchronously without dropping a single frame on the main thread.

Second, we upgraded our database. Up until then, local caching and playlist records had been managed through basic file checks and primitive SQLite queries. For v4.0, we migrated entirely to Android's Room Database framework. Room gave us safe schema migrations, robust compile-time SQL verification, and a structured local caching layer that

meant song lists, favorites, and history loaded up without hesitation. Every time a user saved a track or created a custom playlist, the data was written cleanly to disk, eliminating the corruption bugs that had occasionally plagued earlier builds.

Third, and most importantly, we completely overhauled the playback core. We retired our older audio setup and adopted Google's modern Media3 ecosystem. Media3

gave us native Crossfade capabilities, rock-solid background playback preservation that ignored aggressive battery managers, and clean audio routing so plugging or unplugging wired headphones would not cause awkward delays or app crashes.

A massive shoutout and architectural credit here goes to Echo-Music by @iad1tya. Studying how Echo-Music handled its high-performance audio pipeline

and media handling protocols gave us vital clues on how to structure our Media3 integration properly. Analyzing open-source repositories opened our eyes to how production-grade Android apps actually handle low-level media sessions, buffer states, and background synchronization. It was proof of why open-source software is so powerful, with young developers learning from community-built projects across the globe.

Alongside the visual redesign and core engine upgrades, we revisited one of our most exciting features: Listen Together, powered by our companion platform, BeatLabs.

Listen Together had originally been a rough proof-of-concept allowing two phones on the same local Wi-Fi network to sync audio playback. In earlier builds, one phone would occasionally lag behind the other, turning a shared jam

session into an echoey mess. For v4.0, we completely overhauled the BeatLabs local server engine. We rebuilt the packet sync protocol, lowered latency down to milliseconds, and added instant handshake verification. Now, friends sitting in the same room on the same local network could sync up their playback flawlessly, effectively turning multiple phones into a coordinated speaker setup.

Testing this feature required setting up multiple devices in our rooms. We would start the BeatLabs local instance on a laptop, connect two different phones over our home Wi-Fi, and monitor the playback clock down to the millisecond. Whenever the slave device drifted even slightly off-tempo, the local server sent a lightweight clock sync packet to gently nudge the playback position without causing an audible pop or skip. The moment we heard two phones across the

room playing the exact same drum kick in perfect unison, we knew Listen Together had finally evolved from a glitchy experiment into a killer feature.

To support this expanding ecosystem, we kept our companion websites, our main portal at beatwave.oneapp.dev and the new sandbox network at beatlabs.oneapp.dev, incredibly fast and lightweight. Instead of using heavy web frameworks that

take seconds to load, we built the front-end directly with raw HTML5, modern CSS3, and vanilla JavaScript to guarantee sub-second page loads even on spotty mobile data.

We wanted visitors to click our link and instantly see the documentation, download options, and community channels without waiting for massive bundles to download. The web interface mirrored the Liquid Glass design of the app, featuring

dark slate backdrops,
translucent navigation bars,
and glowing cyan accents
that tied our entire brand
identity together.

Every afternoon from 5:00
PM to dinner time, Akshansh
and I worked methodically
through our task lists. We
tracked bug reports on our
shared digital scratchpad,
ticked off completed
optimizations, and steadily
eliminated every trace of
legacy code. We addressed
edge-case crashes on

budget devices, fixed awkward layout shifts when keyboards popped up, and streamlined the app's overall memory footprint so that running background audio wouldn't choke phones with lower RAM.

By late May, after weeks of intensive coding, designing the new logo, and fine-tuning the Liquid Glass layout, the final pieces of Version 4.0 snapped into place.

We gathered on our voice call, ran our final checks

across various devices, and compiled the release binary: BeatWave-v4.0-Release.apk.

Opening the app felt like a completely new experience.

The freshly designed wave logo greeted us on launch, followed by the seamless Liquid Glass dashboard.

Translucent menus floated effortlessly over softly blurred album artwork, the new Media3 engine handled track transitions with buttery smooth crossfades, and switching themes

transformed the entire interface instantly without a single stutter.

We put together our official release notes, detailing the UI overhaul, the new logo, custom themes, improved Listen Together engine, Room database migration, and Media3 integration. We updated GitHub, refreshed the website download link, posted the update to our Discord community and our Instagram page @_beat_labs, and opened

our support inbox at
beatwave790@gmail.com.

BeatWave v4.0 was not just another incremental update with a handful of bug fixes. With our brand-new logo, the Liquid Glass redesign, and a modern Kotlin and Media3 foundation, the app finally looked and performed like a truly professional audio player. We had outgrown the rough early days, and BeatWave was ready for an even bigger stage.

11

The Double Century: 200 Waves

After we dropped Version 4.0 with the Liquid Glass look and the new logo, the entire atmosphere around BeatWave shifted completely.

Reaching one hundred downloads back on May 7th had felt like surviving an

impossible test. It was an uphill battle where every single installation had to be fought for, argued over, and watched under a microscope. But pushing toward two hundred downloads felt different. It felt like watching something we had built with our bare hands finally pick up its own momentum and start walking into the world on its own legs.

We were no longer in that desperate phase where we

had to go around asking classmates, distant relatives, to tap an unknown link just to make our dashboard look alive. The dynamic had quietly flipped. People were discovering BeatWave without us having to say a single word to them. They were stumbling onto our project repository through GitHub search, reading through our documentation, checking out our showcase clips on Instagram under @_beat_labs, and finding the

direct APK download on our site at beatwave.de5.net

For the very first time since we wrote the first line of code, our database was recording incoming traffic while we were doing absolutely nothing.

You would close your laptop in the evening, sit down with your textbooks to finish school assignments, and when you opened the dashboard the next afternoon, four or five new installations had registered

from cities we had never even visited. It was a strange, thrilling feeling. The application was no longer confined to our local circle or shared as a favor between friends. It was spreading organically because people genuinely liked using it.

A huge part of this sudden acceleration came directly from the changes we had made in Version 4.0.

The Liquid Glass redesign had completely eliminated the rough, prototype vibe

that earlier builds suffered from. Before v4, when someone opened BeatWave, it looked like what it was: an early open-source side project made by students. It had worked, but it had lacked that clean visual polish that makes an app feel trustworthy. With the frosted glass panels, the modern wave logo, and the custom theme engine, the first impression was completely transformed.

Our Discord community, which had started as a tiny room with just 2 people, began growing day by day. Users were dropping into the chat channels and posting screenshots of their home setups, showing off how they customized their player colors to match their wallpaper or sharing their favorite blurred-artwork backgrounds.

Others were testing out the overhauled Listen Together feature through the BeatLabs

local network setup. Seeing teenagers in different places using our local synchronization protocol to listen to the same tracks simultaneously with zero lag was mind-blowing. It proved that all those late afternoons spent tearing down the audio architecture and rewriting everything on top of modern Kotlin, Room Database, and Google's Media3 platform had been worth every ounce of frustration. The nasty stutters were gone, the memory leaks were solved,

and the player was rock solid.

As May rolled over into June, our daily routine settled into an exciting pattern. Every afternoon, the moment 5:00 PM arrived, Akshansh and I jumped onto our usual Discord voice channel.

"Did you look at the numbers before joining the call?" I asked on one of those early June afternoons, putting on my headset and watching Android Studio sit quietly in the background.

"Not yet," Akshansh said, his microphone rustling as he settled into his chair.

"Open the dashboard," I told him, leaning back in my desk chair with a grin.

I heard the sound of his mouse clicking rapidly through his mic, followed by a short silence as the browser tab resolved the database query.

"One hundred and ninety-one," Akshansh said, his tone lifting with pure excitement.

"Bro, we gained nearly forty

downloads since we put up the v4 release notes!"

"Forty downloads in just a few weeks," I said.

"Remember how long it took us to crawl from fifty to eighty-five? We were stuck for days, wondering if anyone would ever care about v3. Now we're gaining steady numbers without even pushing an emergency hotfix."

"Nine left," Akshansh pointed out, highlighting the number on his screen share. "We are

literally nine downloads away from the double century. At this rate, we might cross two hundred before this coming week ends."

"Let's not get ahead of ourselves," I laughed, remembering how tense the countdown to one hundred had been back on May 7th.

"Every time we start celebrating early, the counter decides to freeze for twenty-four hours."

The companion websites had played a massive role in making the whole platform feel legitimate. When someone visited `beatwave.de5.net`, they weren't greeted by an ugly raw folder index or a suspicious file host. They saw a responsive, dark-mode landing page that clearly explained the ad-free philosophy, showed off the Liquid Glass screenshots, provided direct links to the source code, and gave clear instructions on how to install

the APK. It removed all the hesitation that usually comes with installing third-party Android packages.

Over the next couple of days, the countdown to two hundred became our background soundtrack.

Whenever we were at our desks after school, between reviewing pull requests, checking user feedback emails at beatwave790@gmail.com, and answering questions in our Discord server, our eyes

would naturally drift toward the server monitor.

By June 5th, the counter ticked up to 194.

Two of those downloads came through our GitHub repository, where someone had starred the project and shared the direct link on a forum for open-source Android utilities. A developer from abroad had even dropped a kind comment on our commit history, complimenting the clean separation of our Media3

audio service and the lightweight Room database implementation. Reading a compliment from an actual programmer who had taken the time to look through our repository files felt surreal. It reinforced the idea that what we were doing was real engineering, not just a passing hobby.

By the evening of June 6th, before shutting down my laptop to finish my pending schoolwork, I checked the

portal one last time under
the dim light of my room.

197.

Three downloads left.

I leaned closer to the screen,
watching the digits glow
against the dark interface.

Three more users. It was
close enough that someone
waking up in a different time
zone could push us across
the line while we slept. I
turned off the screen, took
off my headset, and lay in
bed for a while, thinking

about how far the project had come.

Just a few months earlier, BeatWave had been nothing more than a shared frustration between two friends talking after an exam, complaining about how mainstream streaming apps were ruining the listening experience with constant commercial interruptions. Back then, we barely knew how to handle Gradle dependencies properly, let alone build a custom audio

pipeline or structure a multi-tier cache. Now, nearly two hundred devices had our software installed in their memory, running code that we had written, tested, and polished right here in our bedrooms.

That brought us to Sunday, June 7th, 2026. Exactly one month to the day after we had celebrated hitting our first hundred downloads.

The day was quiet, warm, and calm. I finished my afternoon schedule,

wrapped up what I needed to do, and walked into my room around 4:50 PM. I powered on the system, opened Discord, and waited for the clock to hit five.

Right on cue, Akshansh's username lit up with a green circle on the voice roster.

"You there?" he asked through the speakers.

"Yeah, I'm here," I said, putting my headset on and adjusting the microphone.

"Did you open the site yet?"

"Not yet," he said. "I wanted to wait until we were both on the call. Today is June 7th, exactly one month since May 7th. Let's see if those three downloads came through."

I navigated to the analytics panel, hit refresh, and let the server query run. The progress bar zipped across the browser tab in half a second, pulling the latest records from the database.

The screen painted the update.

200.

A perfectly even, bold two hundred.

We both looked at the number sitting quietly in the center of the card. Exactly one month after hitting 100 on May 7th, we had reached 200 on June 7th.

There was no chaotic screaming this time. Neither of us slammed our fists on the desk, and neither of our mothers had to come running down the hallway to ask why the house was shaking.

Instead, a quiet, deep sense of satisfaction settled over the call.

"Two hundred," Akshansh said softly, exhaling into his microphone with a slow, genuine laugh. "Look at that, bro. A clean double century. Exactly one month later."

"Two hundred real installations," I replied, leaning back in my chair and staring at the display. "We actually doubled the milestone."

"And think about how we got here," Akshansh said. "When we hit one hundred on May 7th, it felt like an absolute miracle that drained all our energy. We were sitting there counting down single clicks, begging the universe for two downloads to get across the finish line. This time, it just happened naturally. People found the app, liked the redesign, and kept it on their phones."

"Because the product actually holds up now," I

said. "Version 4.0 gave BeatWave its own identity. It doesn't look like an imitation of something else anymore. The Liquid Glass UI, the new logo, the speed of Media3, the fact that you can customize your colors—it feels like a complete package."

We pulled up the detailed analytics tab to see how the downloads were distributed across the different builds. Out of the two hundred total lifetime installations, Version

4.0 had captured a massive share in a fraction of the time compared to v1, v2, and v3 combined. The stability upgrades had practically eliminated user churn; people weren't downloading the APK, encountering a crash, and immediately deleting it anymore. They were keeping it, updating it when we pushed new releases, and using it as their default everyday audio player.

We opened our Discord announcements channel and drafted a short, humble message for our growing community:

Plaintext



200 Downloads
Milestone!

On May 7th, we hit 100.
Today, on June 7th, we
officially crossed 200 total
downloads for BeatWave!

A huge thank you to everyone
who tested the Liquid Glass
redesign, shared their

custom themes, reported bugs, and supported the project on GitHub and Discord.

Two months ago, this was just an experiment between two students. Today, it's a growing community.

More optimizations, features, and refinements are on the way. Thank you for riding the wave with us!

- Aarav & Akshansh

We hit send. Within minutes, reaction emojis started

lighting up beneath the post. Community members dropped congratulatory messages in the general chat, sharing their thoughts on what features they wanted to see next and praising how snappy the latest build felt on their daily drivers.

Seeing those messages roll in made something very clear to both of us. Hitting two hundred downloads wasn't just another number to tick off in a notebook. It was

definitive proof that BeatWave wasn't a one-hit wonder or a brief fluke that was going to fade away after the initial hype.

We had taken an ambitious idea, pushed through build failures, survived devastating lack of traction with earlier versions, handled the stress of school routines, and completely rebuilt our technical stack from the ground up until the software was undeniable.

Two hundred people were carrying our work in their pockets. Two hundred people had chosen our ad-free vision.

Akshansh broke the silence on the call with a quiet chuckle. "So, What do we do now? Take a week off?"

I looked at the glowing wave logo pinned at the top of our documentation, then back at the analytics dashboard where the number 200 sat proudly in the center on June 7th.

"No chance," I said, a wide grin spreading across my face. "Now we see what it takes to hit three hundred."

12

The Flop Era

There is a strange kind of confidence that takes over when things finally start working. On June 7th, 2026, when our counter clicked over to two hundred downloads, Akshansh and I felt completely unstoppable. We had survived the

nightmare of broken builds, pushed past the taunts from classmates, survived the dry spell of Version 3.0, and completely transformed our codebase with modern Kotlin, Room Database, Media3, and the Liquid Glass redesign. We genuinely believed we had cracked the secret formula. We thought that from here on out, every time we dropped a build, the numbers would just double on their own.

We were completely, utterly wrong.

Software development has a brutal way of humbling you the exact second you start getting arrogant. What followed June was not a glorious climb to five hundred or a thousand downloads. Instead, July 2026 became the darkest, most discouraging month in the entire history of BeatWave. It was the month our momentum slammed directly into a concrete wall,

leaving us staring at numbers so low they made us question why we were even doing this in the first place.

It all started in early July.

With schools easing into mid-term routines and our evening schedules relatively stable, Akshansh and I decided it was time to capitalize on the success of Version 4.0. We didn't want the hype to die down. We wanted to keep our community fed with regular improvements, small feature

bumps, and tighter optimizations.

On July 12th, 2026, we officially compiled and pushed out our first summer follow-up: **BeatWave-v4.1-release.apk**.

In v4.1, we focused heavily on low-level media stability, borrowing architectural inspiration from the high-performance pipeline of Echo-Music. We tweaked how the ExoPlayer buffers pre-fetched segments of audio, cleaned up memory

overhead, and eliminated rare visual glitches when rapidly switching between the dark theme and custom user palettes. We wrote a clean changelog, posted the release to our Discord community, put up stories on our Instagram page @_beat_labs, and refreshed the download files on beatwave.de5.net

We sat back on our 5:00 PM Discord call that Sunday evening, waiting for the rush.

"How long do you think it takes to hit fifty on this one?" Akshansh asked, casually looking over his GitHub notifications while we hung out in the voice channel.

"Give it twenty-four hours," I said, leaning back with total ease. "People who have v4.0 are going to update, plus new users coming in from the landing page. It should fly."

Monday came. Tuesday passed. Wednesday rolled by.

The download counter on our release server barely budged. It wasn't flying. It was crawling like a wounded snail.

By the end of the week, the total download count for BeatWave-v4.1-release.apk sat frozen at a miserable, mocking number.

COUNTER: 16 APKS.

Sixteen downloads.

Not sixty. Not one hundred and sixty. Sixteen.

Sixteen downloads across an entire week for an update we had spent days testing and polishing. It was lower than even the disastrous early days of v3.0. The realization hit us like a bucket of ice water. Most of our existing two hundred users didn't even bother updating.

Because we didn't have an automated background updater or an official app store distribution platform, every single update required a user to manually visit our website, download a new

APK file, and install it over the old package.

To an everyday listener who just wanted to hear music while studying or traveling, Version 4.0 was already working fine. They didn't care about background buffer optimizations or low-level memory cleanup. To them, downloading a whole new file for minor tweaks wasn't worth the effort.

"Sixteen," Akshansh muttered during our call that weekend, his voice

completely devoid of the excitement from June. "All that testing for sixteen downloads. My family members alone account for four of those."

"Maybe people just didn't see the post," I said, desperately trying to find an excuse. "The Instagram algorithm barely showed our release story. And our Discord only has thirty active people. If we release a proper universal build with wider device support and

layout fixes, it will pick back up."

Instead of stepping back, taking a breath, and analyzing why people weren't downloading the file, we panicked. And when developers panic, they do the worst thing possible: they rush out another update.

We convinced ourselves that the problem was compatibility. We thought maybe users on different Android CPU architectures were running into installation

parse errors with the release build. So we spent the next six days frantically reworking our Gradle packaging rules, generating a unified multi-architecture package that could run on anything from budget ArmV7 devices to modern 64-bit phones. We also tweaked layout scaling for compact screens and squashed a minor UI glitch where the seekbar thumb occasionally clipped outside the frame.

On July 19th, 2026—just one week after the failure of v4.1—we launched:

RELEASE v4.2 (BeatWave-v4.2-Universal.apk).

We threw everything we had into announcing it. We pinned the message in our Discord announcements, redesigned the banner on beatwave.de5.net, and messaged our friends directly asking them to check out the universal build.

"This is the one," I told Akshansh on Sunday night

after the upload completed.
"Universal support solves the friction. If anyone had install errors before, this fixes it completely."

The universe, once again,
had other plans.

Monday morning: two
downloads.

Tuesday: zero.

Wednesday: three.

Thursday: one.

By the time the weekend
arrived, the server logs
stopped updating entirely.

The dashboard stared back at us with the cruelest verdict yet:

COUNTER: 12 APKS.

Twelve downloads.

We had somehow managed to do *worse* than the previous week. From sixteen downloads down to twelve. It felt like a punch straight to the gut. We had spent hours sitting at our desks after school, ignoring our leisure time, skipping evening gaming sessions, and staring at terminal output, only to

produce a build that literally twelve human beings on planet Earth cared to install.

The silence on our Discord call that evening was deafening. There was no upbeat chatter, no jokes about our teachers, and no excitement.

"Twelve" Akshansh said, his voice flat, sounding completely burnt out. "We went backwards. Twelve downloads means nobody is paying attention anymore."

The wave is completely dead."

"I don't understand," I said, staring blankly at the dark interface of our site. "We have over two hundred total downloads. Where did everyone go? Why did they stop caring?"

"Because we are spamming updates nobody asked for," Akshansh answered bluntly. "Think about it from their perspective. Who wants to manually download and install a new thirty-megabyte

APK every seven days just to read that we fixed a seekbar glitch and updated compiler rules? We're treating this like a major enterprise release cycle, but our users just want an app that plays songs without bothering them."

His words stung because they were completely true, but our stubbornness hadn't completely burned out yet. We were stuck in a toxic loop that so many young creators fall into: when your project starts failing, you feel like the

only way to save it is to force another release. You tell yourself that the *next* update will be the magic one that brings everyone back.

Without waiting, without taking time to rethink our strategy, we threw together **BeatWave-v4.3-release.apk**.

We didn't even have major new features to offer. Version 4.3 was essentially a collection of small layout adjustments, tiny animation timing tweaks, and minor

dependency upgrades. We pushed it out quietly near the end of the month, half-hoping that by some miracle, our analytics graph would suddenly spike vertically and rescue us from the embarrassment.

It didn't.

Version 4.3 flopped harder than anything we had ever touched. The download counter crawled into single digits, struggling to even match the twelve downloads of v4.2. Across three full

releases—v4.1, v4.2, and v4.3—our total combined downloads barely cleared thirty-five installations.

Thirty-five downloads across three separate releases.

It was an undeniable, humiliating disaster.

The excitement of June 7th felt like a distant, cruel memory. Back then, we had proudly posted our 200-download milestone, talking about reaching five hundred and dreaming of a thousand. Now, looking at our empty

traffic charts, those dreams felt ridiculous. We felt like impostors—two eighth-graders who had gotten lucky once with a viral link and now had run out of road.

The mental toll of that dry spell was exhausting. When you put your heart into a project, low numbers don't just feel like bad analytics; they feel like a personal rejection. Every time we opened the website and saw the counters sitting motionless at 16 and 12, it

felt like the world was telling us that BeatWave had run its course.

Our school routine didn't help either. Sitting in Section A through long morning lectures, watching the ceiling fan spin lazily under the afternoon heat, I found myself wondering why we were even bothering.

Classmates were still playing games on their phones, listening to whatever mainstream streaming services they already had,

completely unbothered. Nobody in the hallways was talking about BeatWave anymore. The novelty had worn off. We were no longer "the kids who built an app." We were just two students falling behind on revision assignments while maintaining a graveyard of APKs that nobody was downloading.

During our twenty-minute lunch breaks, when I met Akshansh , completely different from the energetic

strategy sessions we used to have. We ate our food in near silence.

"Are we even going to work on it today?" I asked him one afternoon as we packed our empty tiffins.

Akshansh shrugged, "I don't know, man. What is there to work on? We push code, we build the APK, we upload it, and twelve people click it. My laptop fan makes more noise compiling Gradle than the entire userbase makes using

the app. I think people just moved on."

That was the hardest pill to swallow: the idea that people had simply moved on.

When you are thirteen years old, you expect failure to be loud. You expect an app to fail because of a catastrophic bug, a public takedown, or a massive server crash that everyone talks about. You don't expect failure to be quiet. You don't expect it to look like a server log where the timestamp just

stops updating, or a Discord chat where the last message was sent four days ago.

The flop of versions 4.1, 4.2, and 4.3 stripped away all our naive illusions. It proved to us that building good software is only half the battle; understanding your users, respecting their time, and knowing when *not* to push an update is just as critical. We had drowned our own project in unnecessary, minor releases that nobody asked for, and in doing so, we

had killed our own momentum.

One evening toward the end of July, I sat at my wooden desk without turning on Android Studio. The room was dark, lit only by the soft glow of the browser window displaying our release repository.

Plaintext

BeatWave-v4.3-release.apk -
Counter: Low

BeatWave-v4.2-
Universal.apk - Counter: 12

BeatWave-v4.1-release.apk -
Counter: 16

Those three lines looked like tombstones sitting on our screen.

For the first time since that evening after our final exams when we had dreamed up the idea of BeatWave, I genuinely considered shutting down the server and walking away. It felt like the wave had finally reached the shore, crashed against the rocks, and pulled back into

the ocean, leaving nothing behind.

Yet, deep down, underneath all the disappointment and the bruised ego, a tiny stubborn spark refused to die completely.

We had built this app from absolute zero knowledge. A disastrous month and three flopped builds were painful, but they were only fatal if we gave up right there on the floor.

The flop era had broken our arrogance. But it also forced

us to ask the one question that would determine the entire future of BeatWave:

Were we going to let sixteen downloads define the end of our story, or were we going to figure out what real developers do when everything falls apart?...

Little did we know that one post threads will change our entire life.

13

The Viral Thread

By the beginning of August, the silence around BeatWave was almost suffocating. The disastrous release cycle of versions 4.1, 4.2, and 4.3 had drained every single drop of confidence we had built up over the summer. Seeing download numbers like

sixteen, twelve, and single digits on our dashboard had made us feel like our project was officially running on empty. We barely talked about updates anymore. We joined our usual 5:00 PM Discord calls, but instead of compiling builds or testing layouts, we just sat in the voice room with our microphones muted, browsing the web or doing homework in total quiet. There is nothing quite as depressing for a developer as

an empty analytics dashboard. When we were writing the early builds, every failure was loud and obvious. A red error message in the console or a crash dialog on our test phone was something tangible to fight against. But the silence of the flop era was different. It meant that nobody was even checking whether our code worked or not. It felt like we were shouting into a void where nobody was listening.

Then came a completely random Tuesday evening that flipped everything upside down.

I was sitting at my desk, slouched in my wooden chair, scrolling aimlessly through Instagram on my phone. The feed was the usual mix of tech memes, school reels, and gaming clips. Then, as I scrolled past an explore tile, a post caught my eye. It was a screenshot of a Threads post from an independent developer

showing off a minimal open-source audio streaming tool they had built over a weekend.

The numbers beneath the post made me sit up straight.

It had thousands of likes, hundreds of reposts, and a comment section packed with people asking where to get the link, how to compile the source code, and whether it supported background playback. The creator had not run paid ads or reached out to tech

influencers. They had not built a complex marketing strategy. All they had done was write a short, honest thread about being an independent developer building something cool from scratch, complete with a clean thirty-second screen recording showing the interface in action.

I stared at the screen for ten seconds, my heart suddenly beating faster.

I grabbed my headset, pulled the mic down to my mouth, and unmuted on Discord.

"Akshansh, are you there?" I asked.

A rustle came through the speaker. "Yeah, I'm here. What's up?"

"Open Instagram right now," I said, copying the link and pasting it into our general text channel. "Look at this thread."

I heard his mouse click as he opened the link. For about

half a minute, neither of us said anything.

"Over eight thousand views on the video," Akshansh muttered, his voice waking up from the boredom of the last three weeks. "And look at the comments. People are literally begging for the GitHub repo link."

"Look at the interface they're showing," I said, leaning closer to my screen. "It's clean, but it's a basic flutter wrapper with standard cards. We have the entire Liquid

Glass UI. We have dynamic color extraction, custom themes, offline caching, and the YouTube URL parser in Version 4.3. We spent weeks killing ourselves to build all of that, and nobody knows it exists because we've only been talking inside our own tiny thirty-person bubble."

"We've been talking to a wall," Akshansh realized.

"Every time we post an update, we post it to our own Discord server where everyone already has the

app, or to our website where nobody goes unless they already know the URL. We never actually put BeatWave in front of real tech communities."

"Let's write a thread," I said.

The idea was simple, but after weeks of feeling invisible, it felt like a massive leap. We didn't want to sound like a corporate advertisement or two kids begging for sympathy. We wanted to tell the real story: two young students in India

building an ad-free music streaming platform with modern tech, struggling through strict school rules where phones are banned, sketching architecture on paper during short breaks, and rebuilding an entire player engine on top of Kotlin and Media3.

We opened a shared text document and spent an hour drafting the post. We kept it raw, honest, and direct. We talked about why we started BeatWave—the pure

frustration of having music interrupted by endless commercial breaks. We explained how we designed the Liquid Glass aesthetic so the player looked modern and floating. We added a screen recording showing the frosted glass panels adapting in real time to the colors of a song's album art, followed by someone pasting a YouTube URL into the search bar and the audio starting instantly without lag.

At the very end of the thread, we dropped our links: the project repository on GitHub, our community on Discord, and the direct download portal at beatwave.de5.net.

At 7:15 PM, we took a deep breath.

"Ready?" Akshansh asked over the call.

"Hit publish," I said.

He tapped the button. The thread went live.

For the first thirty minutes, nothing out of the ordinary

happened. The post sat there with four likes—one from Akshansh, one from me, one from my sister, and one from a classmate.

"Well, at least we tried," Akshansh said with a dry laugh. "I'm going to eat dinner. Don't stay up staring at an empty feed."

"Yeah, see you tomorrow," I replied, closing the app on my phone and shutting down the laptop.

I went out to the kitchen, had dinner with my family, helped

clear the table, and sat down at the dining table to finish off my school revision. By 10:30 PM, I had completely forgotten about the post.

Right before turning off the lights in my room to go to sleep, I picked up my phone to set the morning alarm.

The screen was completely flooded with notifications.

Not four. Not ten. The notification tray was scrolling endlessly. Threads notifications, Instagram mentions, GitHub star alerts,

and Discord join pings were landing on the screen every few seconds.

My thumb shook slightly as I unlocked the phone and tapped the app icon.

The thread had been picked up by the algorithm. It wasn't just sitting in front of local users; it had reached feeds across India, Southeast Asia, and Europe. The view counter on our screen recording had crossed forty thousand views. The

comment section was
exploding with messages:

*"Wait, two students built
this?"*

*"The Liquid Glass design
looks better than actual
commercial apps."*

*"Is this open-source? Starred
the repo!"*

*"Where can I download the
APK? The link in bio is getting
hammered."*

"Holy crap," I whispered
aloud in the dark room.

My head was spinning with adrenaline, but with a full school day ahead in just a few hours, I forced myself to shut my eyes and sleep.

The next morning, the very second I woke up for school, I grabbed my phone and messaged Akshansh in all caps:

"AKSHANSH WAKE UP. OPEN THE THREAD RIGHT NOW."

His reply came back within ten seconds: *"WHAT IS HAPPENING?! MY DISCORD WON'T STOP RINGING!"*

I hurried to open our website on my mobile browser. When the page finished loading, my eyes locked straight onto the big bold number in the center of the analytics card:

300.

Three hundred downloads.

We were staring at our screens in pure shock. After weeks of brutal silence and flatlined charts, our work had finally broken through the wall. But the sudden surge came with a massive technical reality check. Our

modest static backend and direct file links were groaning under the weight of concurrent traffic. Hundreds of users were trying to pull streaming endpoints, query metadata, and download the installation build all at once.

When I met Akshansh briefly in school during the passing period, his eyes were wide with a mix of shock and stress.

"Our setup is choking," he whispered quickly, keeping an eye out for teachers

patrolling the hallway. "The current file-hosting and static link endpoints are throwing timeouts. If we keep serving hits through these unmanaged routes, the whole site will stop responding."

We realized our old setup had been built for a handful of classmates, not a sudden flood of users from across the internet. If someone visited the site and the download timed out, or if the metadata queries threw a

504 gateway error, they would simply close the tab and forget about BeatWave forever. We had one shot at keeping these visitors, and our infrastructure was on the edge of collapsing.

All through class, my mind was racing, waiting for the final bell to ring.

At 4:00 PM, the moment I reached home, I kicked off my shoes, threw my backpack onto the sofa, and opened the website.

400.

Another hundred downloads had landed while we were sitting in classrooms.

I instantly fired a message to Akshansh: *"BROOOOOO WHAT IS HAPPENING OUR APP GOT 100 NEW DOWNLOADS IN LESS THAN A DAY!"*

He replied within seconds: *"YESSSS I HAVE SEEN IT OUR HARDWORK FINALLY PAID OFF!"*

We spent the afternoon juggling messages, answering bug reports, and

posting thank-you stories across our social channels. We even posted another follow-up thread, sharing more behind-the-scenes details of how we built the player.

The momentum didn't slow down. The next morning, I opened the site half-expecting the wave to have crested around 450.

Instead, the screen read:
500.

I stared at the number, completely speechless. I couldn't believe it. Only Akshansh truly knew how much this meant after the heartbreak of July. We had gained three hundred brand-new users in just two and a half days.

I messaged Akshansh immediately. His reply came back: *"A PROJECT MADE BY TWO TEENS HAS NOW CROSSED OVER 500 DOWNLOADS!"*

We had promised each other earlier that we would throw a small celebration party the day we reached 500 downloads. But the milestone had blown past so fast that we had to reschedule our celebration target to 700.

BeatWave was no longer just a school project. It was reaching people worldwide. Listeners from different countries were submitting feedback, reporting small edge-case bugs, and sending

feature suggestions. Seeing that global support filled us with energy.

Over the next few days, the counter settled around 600. A few weeks earlier, a plateau would have worried us, but now we understood how discovery worked.

Threads was our bridge to the world. We put together another detailed post, this time highlighting the polished Liquid Glass user interface, custom color

theming, and snappy navigation.

That post blew up immediately.

It gathered over 5,000 views in just five hours, bringing in more than a hundred new downloads in a single sixty-minute stretch. Back when we celebrated gaining a hundred downloads over an entire month, we thought that was peak success. Now we found ourselves laughing at our own past doubts, wondering why we had

waited so long to share our journey publicly.

The real climax arrived on an ordinary afternoon when we got back home from school.

I sat down at my desk, booted up my laptop, and loaded beatwave.de5.net for the daily check.

The screen updated, and a giant number hit my eyes:

900.

Nine hundred downloads.

My heart jumped into my throat. I clicked into our

Discord voice channel,
called Akshansh, and
screamed straight into the
microphone: "WE REACHED
900
DOWNLOADSSSSSSSSSSSS!"

Akshansh screamed back so
loud his microphone clipped:
"YESSSSSSSSSS WE DID IT!
WE ARE ONLY 100 AWAY
FROM OUR FIRST MAJOR
MILESTONE!"

We were laughing, yelling,
and talking over each other in
pure disbelief. From sitting at
twelve downloads in the

depths of the flop era to standing at nine hundred active installations, the transformation felt like a dream.

We immediately drafted a celebration thread to mark the 900-download mark, thanking everyone who had starred the repository, joined our Discord, and shared BeatWave with their friends. Just like the posts before it, the celebration thread began picking up likes and reposts within minutes.

Sitting in my room as the evening sun faded, watching the community comments roll in, the mood was pure joy. The doubt, the teasing from classmates, and the exhaustion of balancing code with school had all led to this. BeatWave was thriving, the wave was bigger than ever, and that magical four-digit goal of 1,000 downloads was finally within our grasp.

14

One Thousand: The Promised Party

Nine hundred downloads had felt like standing at the base of the final summit. When you start with zero, sitting in an empty bedroom with nothing more than an open code editor and a rough idea sketched on ruled notebook paper, one

thousand feels like an imaginary number. It belongs to established applications made by funded companies, not two eighth-graders trying to finish their chemistry assignments before a 5:00 PM Discord call.

Yet, as the second week of August unfolded, the counter on beatwave.de5.net kept moving. It had ceased to be a slow, painful grind where we tracked individual IP hits from classmates. The viral threads had turned our tiny

side project into a self-sustaining wave.

Whenever we met at school, the entire mood between Akshansh and me had transformed. The awkwardness from the flop era—when we had avoided eye contact near the lockers out of pure embarrassment over single-digit downloads—was completely gone.

Now, when the bell rang for the five-minute transition between periods and our

paths crossed near the central stairwell connecting the second floor of Section A to the third floor of Section D, we didn't have to whisper about failed builds or broken links. We simply held up our fingers to signal the latest tens count.

"Nine forty when I left home," Akshansh muttered as he passed me with his math register tucked under his arm.

"Nine sixty by the time my bus reached the gate," I

whispered back, grinning as a corridor monitor turned the corner.

We kept walking, but the excitement carried us through the rest of the school day. Knowing that dozens of strangers across different cities were actively installing our APK while our history teacher lectured about medieval treaties made the hours fly past.

When the final 2:55 PM dispersal bell echoed across the campus, the race was

on. I didn't linger by the benches or wait for the hallways to clear. I packed my pens, zipped up my bag, and hurried straight toward the bus lines with my sister.

The ride home through the mid-afternoon traffic felt electric. The roads were jammed with horns, auto-rickshaws, and cycle carts, but my eyes were locked onto my wrist watch, counting down the minutes until I could sit in front of the terminal.

I reached home, kicked off my shoes, and skipped the usual post-school routine. I pulled my laptop out, hit the power button, and threw my headset on.

At 4:45 PM, fifteen minutes before our usual schedule, my Discord pinged.

Akshansh was already online, his status glowing bright green.

"You seeing this?" he asked the moment I joined the voice channel.

"Opening the dashboard right now," I replied, typing in the URL.

The screen flashed, the server queried the database, and the large numeric display rendered on the page:

988.

Twelve downloads remaining.

"Twelve people," Akshansh said, his voice trembling slightly with nervous anticipation. "We're literally

twelve installations away from four digits."

"Remember back in July when we promised each other a party at five hundred downloads?" I laughed, leaning back in my chair. "We blew past five hundred so fast in forty-eight hours that we didn't even have time to celebrate. Then we pushed the party to seven hundred, and seven hundred blew past while we were studying for our weekly unit test."

"No more pushing the goalpost," Akshansh declared firmly over the mic.

"The second that counter hits one thousand, we celebrate. The promised party happens tomorrow right at school. We'll bring treats and do it properly during recess."

We sat side by side on the call, staring at the live logs. In the bottom right corner of my desktop, the system clock ticked steadily: 4:52 PM... 4:56 PM... 5:00 PM.

At 5:03 PM, the dashboard refreshed automatically.

994.

Six remaining.

"Six downloads!" Akshansh yelled, the audio clipping through his headset. "Aarav, hit refresh again!"

"The script auto-polls every sixty seconds, don't spam the server!" I warned him, though my own hand was hovering over the F5 key, practically shaking.

Those next few minutes stretched out endlessly. It brought back memories of May 7th when we had waited for the hundredth user, but the feeling this time was completely different. Back then, hitting one hundred was about survival—it was about proving that we weren't fools wasting our time on an impossible dream. Hitting one thousand was pure triumph. It was proof that two thirteen-year-olds with strict school hours, no commercial budget, and

locked-away smartphones could build software that reached an audience of four figures.

At 5:08 PM, the page blinked.

998.

"TWO LEFT!" I shouted, gripping the edges of my wooden desk. "TWO MORE!"

"Who is downloading it right now?!" Akshansh yelled back. "Look at the location logs!"

"One from Bangalore, one from Germany!" I read off the

terminal. "It's completely global!"

We stopped talking. The room went dead silent except for the low whir of my laptop fan spinning at maximum speed.

The digital clock in the corner of my taskbar flipped to 5:10 PM.

The auto-refresh indicator flashed at the top of the browser tab. The central card cleared for a fraction of a second as the backend

tallied the newest incoming records.

Then, the digits repainted onto the glass-styled interface:

1,000.

A clean, solid, four-digit number. One comma zero zero zero.

For three seconds, neither of us said a word. We just stared at the screen, letting our eyes adjust to the extra digit that had never been there before.

Then our voice channel
exploded into sheer,
unfiltered chaos.

"ONE THOUSAND!"

Akshansh roared at the top of
his lungs, his voice
completely breaking through
the audio feed. "WE DID IT!
ONE THOUSAND
DOWNLOADS!"

"FOUR DIGITS, BRO! FOUR
DIGITS!" I screamed back,
throwing my hands in the air
and knocking my notebook
off the side of the desk. "WE

OFFICIALLY CROSSED ONE THOUSAND!"

We were laughing, shouting, and cheering over each other like madmen. Months of struggle, the painful crashes of the early alpha builds, the humiliation of the flop era when versions 4.1 and 4.2 had stalled out, the late-night sprints to overhaul the architecture on Kotlin, Room, Media3, and Convex—every single late night and anxious lunch break culminated in that one single number

glowing quietly on the screen.

I sat back in my chair, staring at the monitor with a wide, breathless grin.

"One thousand real downloads," I said into the mic, my voice finally settling down. "We actually made it happen, Akshansh."

"Four digits, Aarav," he replied, letting out a deep sigh of relief. "Remember the deal. Tomorrow is Friday. The promised party happens at

school. Pack the good stuff in your bag."

"You already know it," I laughed. "See you in the morning."

The next day, Friday, arrived with an entirely different kind of energy. Usually, Friday mornings were defined by the dread of end-of-the-week tests and heavy bags, but today, I walked through the school gates with a light step. Tucked safely at the bottom of my backpack, alongside my notebooks,

was a stash of chocolates, packets of chips, and extra snacks bought specifically for the milestone.

During the morning periods in Section A, it was almost impossible to pay attention to the whiteboard. Every time the clock ticked closer to noon, the anticipation grew.

Finally, the long-awaited 11:50 AM recess bell rang out across the building.

The classroom door swung open, and the usual stampede of students

poured into the hallway.
Instead of heading outside, I
grabbed my lunchbox,
reached into my bag for the
extra snack packets, and
walked quickly down the
corridor toward Section D.

Akshansh was already
standing outside his
classroom door, a massive
grin spread across his face,
holding his own tiffin box
packed with treats.

"One thousand," he said,
holding up both hands to give
me a loud high-five that

echoed right down the hallway.

"One thousand, bro," I laughed.

We ducked inside the classroom to our desks, pulling together two wooden benches so we could sit together. We popped open our lunchboxes, spread out the snacks, opened the packets of chips, and divided the chocolates across the desk. A few close classmates from our section stopped by,

curious about the sudden feast.

"What's the occasion today?" one of them asked, grabbing a handful of chips from the table. "Someone's birthday?"

"No birthday," Akshansh replied proudly, leaning against the bench.

"BeatWave just crossed one thousand downloads."

The kids looked at us in surprise. Back in May, some of these same students had teased us when our test

builds were stuttering and buggy. Now, they were standing around our desk, genuinely impressed, asking how many different countries people were downloading it from and how we managed to pull it off while attending the same classes as them.

Sitting right there on the school benches, surrounded by our friends, eating snacks, and laughing about the journey, the promised party finally happened. It wasn't in a restaurant or on a stage; it

was right inside the classrooms where the idea had started as scribbles on the margins of rough registers.

We remembered the very first build of Version 1.0, when the app had crashed on startup because of missing permissions. We laughed about the panic during lunch breaks when our layouts broke. We remembered the brutal silence of July when Version 4.2 got only twelve

downloads and we had nearly decided to delete the repository and give up entirely.

"If we had quit during the flop era," Akshansh said, unwrapping a chocolate, "we never would have written that thread. We never would have migrated to Convx. We never would have reached 4.3.5."

"Quitting is easy," I replied, looking around the noisy, buzzing classroom. "Pushing through when nobody cares

is the only thing that actually counts."

The warning bell echoed through the school building, signaling the end of recess. Students began throwing wrappers into the dustbin and heading back to their assigned seats. Akshansh and I packed our empty lunchboxes and wiped down the bench.

"Period five is starting," Akshansh said, giving me a nod as he picked up his bag.

"Ready for the rest of the day?"

"Always," I smiled.

I walked back to Section A feeling lighter and more determined than ever. The double life of being a student by day and a developer by night was demanding, exhausting, and full of obstacles. But as I took my seat and opened my notebook, one thing was absolutely clear:

The wave wasn't stopping anytime soon. BeatWave had

reached one thousand, and
our hardwork payed of!!!

Remember BeatWave Was
the best, is the best, and will
be the best

Peace out !!!!

By the devs of- BeatWave

Aarav Sharma

Akshansh Sinha

Aarav Singh



Check out our website by
scanning this!!



Check our github repo here!

